



Group 2 Presentation Data Mining of Customer Churn for Telco

Cedric Tong, Daniel Graham, Justin Gourneau,
Tyler Barton, Yashin Rodriguez

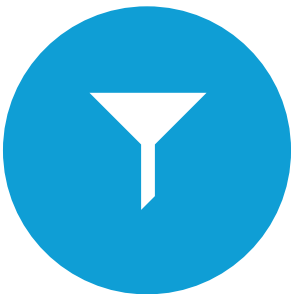
Project Context



Data sourced from IBM Sample Data Sets via Kaggle



Generated data represents client information for a fictional TelCo company providing internet and phone service



Dataset consists of tuples of customers, details for each customer and our target variable, churn



Churn – Whether or not a customer leaves the company



Business Understanding

- Our company provides phone and internet service
- Records some biographical data
 - Gender, Partner, Dependents, Senior Citizen, etc.
- As well as details regarding their contract and history with the company
 - Tenure, Payment Method, Internet Service, Phone Service, Monthly Charges, etc.
- Records whether or not a customer has churned, or left the company
- Customer acquisition is expensive and a churning customer represents a loss in revenue
- We should seek to prevent churn

Business Objective



Develop a model which can predict which customer will churn



Use the model to identify traits that cause a customer to churn



Use the model to direct customer retention efforts at customers most likely to churn

Success Criteria

Data mining will be considered successful if

- Provides meaningful insights into factors contributing to customer churn
- Successfully identifies customers at being at risk of churning

Successful outcomes should

- Provide recommendation of action to support customer retention
- Identify features which may not be competitive or suit customer needs
- Allow allocation of customer retention services to those most in need

Successful model

- Our initial intention is to provide a model which predicts churning customers with an accuracy of 80%

Dataset in Detail

- Origin
 - <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
 - Adapted from IBM sample data set
 - Generated data with no personally identifiable information
- Shape
 - 7,043 rows with 21 fields
- Quality
 - All rows unique
 - No missing fields, some empty strings as placeholders for 0 imputed as such

Dataset in Detail

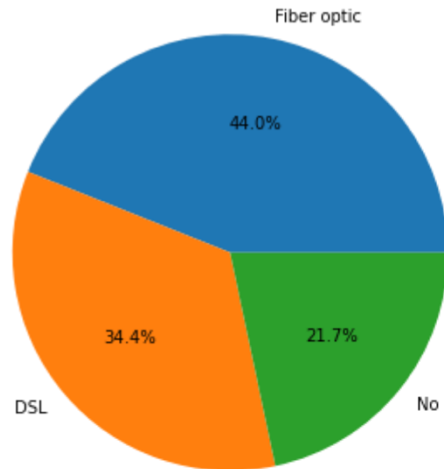
- Dataset Type
 - Tabular
- Nominal Features
 - CustomerID, gender, SeniorCitizen, Partner, Dependents, PhoneService, InternetService, Churn etc.
- Numerical
 - TotalCharges, MonthlyCharges, tenure

Field Name	Data Type	Attribute Type
<u>customerID</u>	string	nominal
<u>gender</u>	string	nominal
<u>SeniorCitizen</u>	boolean	nominal
<u>Partner</u>	string (boolean - "yes/no")	nominal
<u>Dependents</u>	string (boolean - "yes/no")	nominal
<u>tenure</u>	integer	discrete ratio-scaled
<u>PhoneService</u>	string (boolean - "yes/no")	nominal
<u>MultipleLines</u>	String (boolean - "yes/no")	nominal
<u>InternetService</u>	string	nominal
<u>OnlineSecurity</u>	string (boolean - "yes/no")	nominal
<u>OnlineBackup</u>	string (boolean - "yes/no")	nominal
<u>DeviceProtection</u>	string (boolean - "yes/no")	nominal
<u>TechSupport</u>	string (boolean - "yes/no")	nominal
<u>StreamingTV</u>	string (boolean - "yes/no")	nominal
<u>StreamingMovies</u>	string (boolean - "yes/no")	nominal
<u>Contract</u>	string	ordinal
<u>PaperlessBilling</u>	string (boolean - "yes/no")	nominal
<u>PaymentMethod</u>	string	nominal
<u>MonthlyCharges</u>	float	discrete ratio-scaled
<u>TotalCharges</u>	float	discrete ratio-scaled
<u>Churn</u>	string (boolean - "yes/no")	nominal

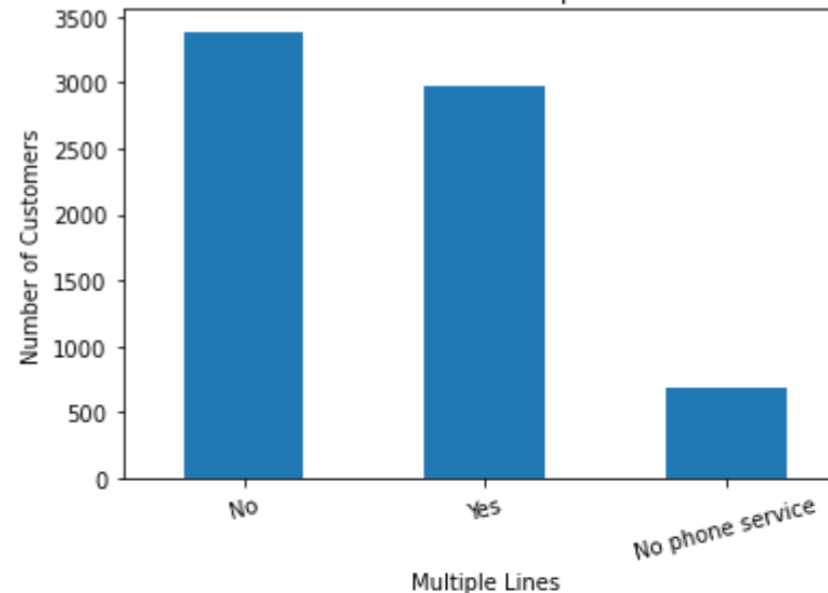
Data Exploration

- CRISP-DM: Data Understanding
 - Explore the dataset before modeling
- Goal: Identify which variables may be useful to predicting churn
 - Used visualizations, statistics, and correlation analysis

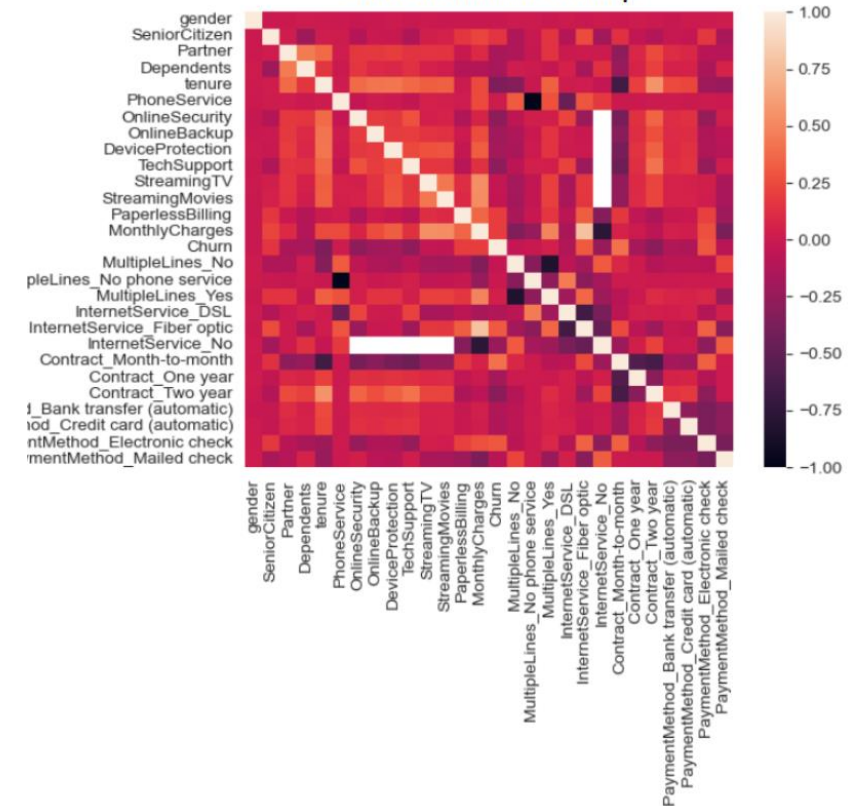
Distribution of InternetService



Distribution of MultipleLines

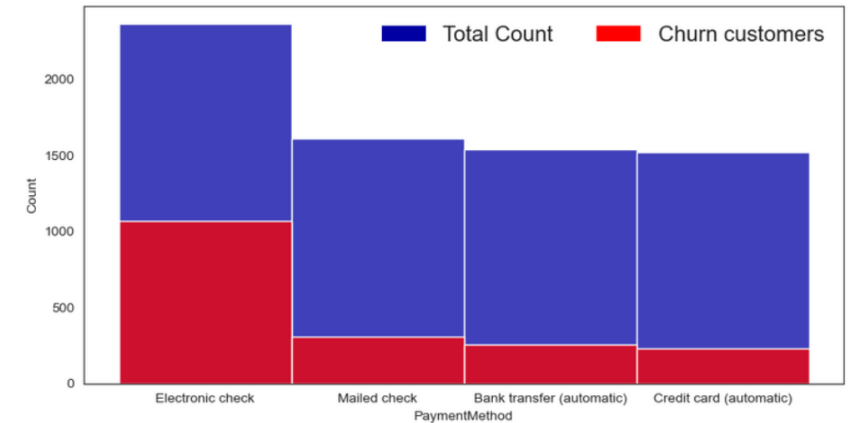


Correlation Heat Map



Feature Descriptions

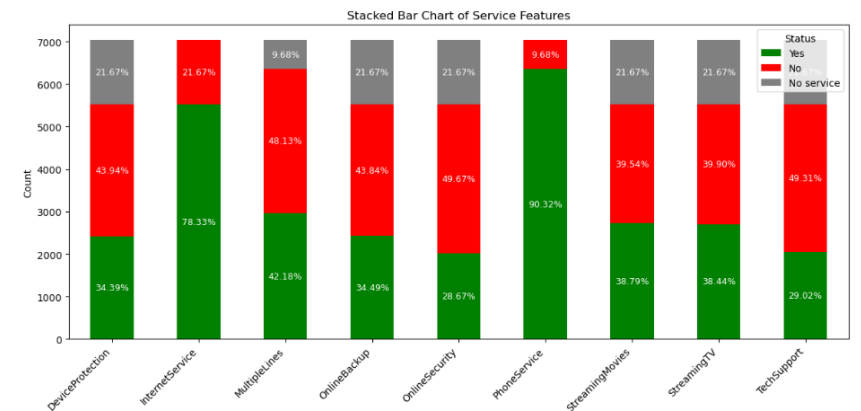
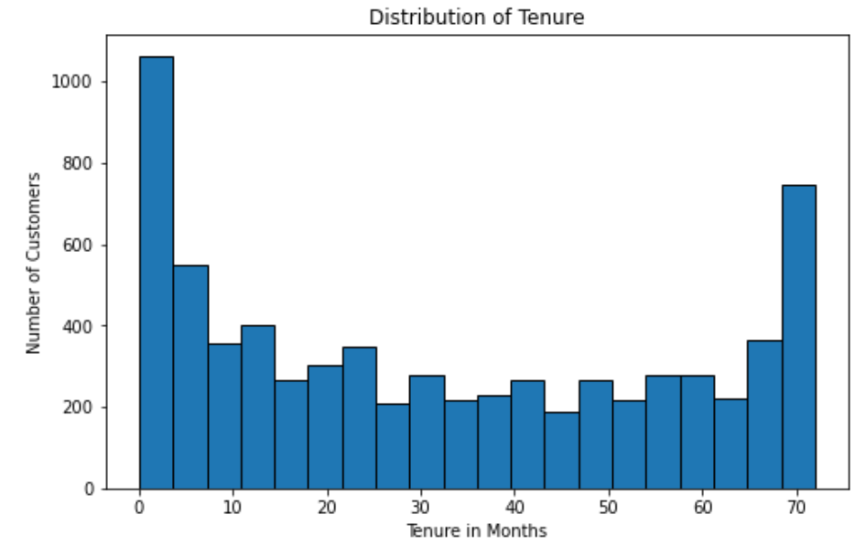
- Target variable: Churn
 - No churn: 5,174 customers, 73.46%
 - Yes churn: 1,869 customers, 26.54%
 - Class Imbalance
- PaymentMethod showed different churn patterns depending on how the customers paid
 - Electronic check was positively correlated to churn



	Bank transfer (automatic)	Credit card (automatic)	Electronic check	Mailed check	Churn
Bank transfer (automatic)	1.000000	-0.278215	-0.376762	-0.288685	-0.117937
Credit card (automatic)	-0.278215	1.000000	-0.373322	-0.286049	-0.134302
Electronic check	-0.376762	-0.373322	1.000000	-0.387372	0.301919
Mailed check	-0.288685	-0.286049	-0.387372	1.000000	-0.091683
Churn	-0.117937	-0.134302	0.301919	-0.091683	1.000000

Key Relationships

- Tenure: wide range from 0 to 72 months with a concentration at both extremes
 - Negative correlation with churn
- InternetService showed a visible relation with churn
 - Churn patterns varying by service type
- Any correlation with churn means it will be useful as a predictor variable for churn



```
df['TotalCharges'] = df['TotalCharges'].replace(' ', 0)
df['MonthlyCharges'] = df['MonthlyCharges'].replace(' ', 0)
df['tenure'] = df['tenure'].replace(' ', 0)
df['TotalCharges'] = df['TotalCharges'].astype(float)
df['MonthlyCharges'] = df['MonthlyCharges'].astype(float)
df['tenure'] = df['tenure'].astype(float)
```

Data Preprocessing

Data Cleaning

- Missing values need imputing
 - Handful of fields with missing values (a single whitespace). This was associated with new customers.
 - Safe to impute these columns to a value of 0 and ensure their types are float (for later quantitative analysis).

Data Preprocessing

Data Transformation

- Categorical values need encoding.
 - Many binary categorical values like Male/Female and Yes/No across multiple columns can be normalized to numeric zeros and ones (0 / 1).

```
1 le = LabelEncoder()
2 enc = df[["gender", "Partner", "Dependents", "PhoneService", "PaperlessBilling", "Churn"]]
3
4 for c in enc.columns:
5     enc[c] = le.fit_transform(enc[c])
6
7 enc.head(10)
```

	# gender	# Partner	# Dependents	# PhoneService	# PaperlessBilling	# Churn
0	0	0	1	0	0	1
1	1	1	0	0	1	0
2	1	1	0	0	1	1
3	1	1	0	0	0	0
4	0	0	0	0	1	1
5	0	0	0	0	1	1
6	1	0	0	1	1	0
7	0	0	0	0	0	0
8	0	0	1	0	1	1
9	1	1	0	1	1	0

10 rows x 6 cols 10 per page

Page 1 of 1

Data Preprocessing

Data Transformation (cont.)

- Multiclass categorical features need encoding
 - We one hot encoded multiclass categorical features across a small number of highly important fields.

```
category_columns = [
    "MultipleLines",
    "InternetService",
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies",
    "Contract",
    "PaymentMethod"
]

df_encoded = pd.get_dummies(df, columns = category_columns)

df_encoded
```

StreamingMovies_No	StreamingMovies_No internet service	StreamingMovies_Yes	Contract_Month-to-month	Contract_One year	Contract_Two year	PaymentMethod_Bank transfer (automatic)	PaymentMethod_Credit card (automatic)
1	0	0	1	0	0	0	0
1	0	0	0	1	0	0	0
1	0	0	1	0	0	0	0
1	0	0	0	1	0	1	0
1	0	0	1	0	0	0	0
...	...	...	...	...	...	...	...
0	0	1	0	1	0	0	0
0	0	1	0	1	0	0	1
1	0	0	1	0	0	0	0
1	0	0	1	0	0	0	0
0	0	1	0	0	1	1	0

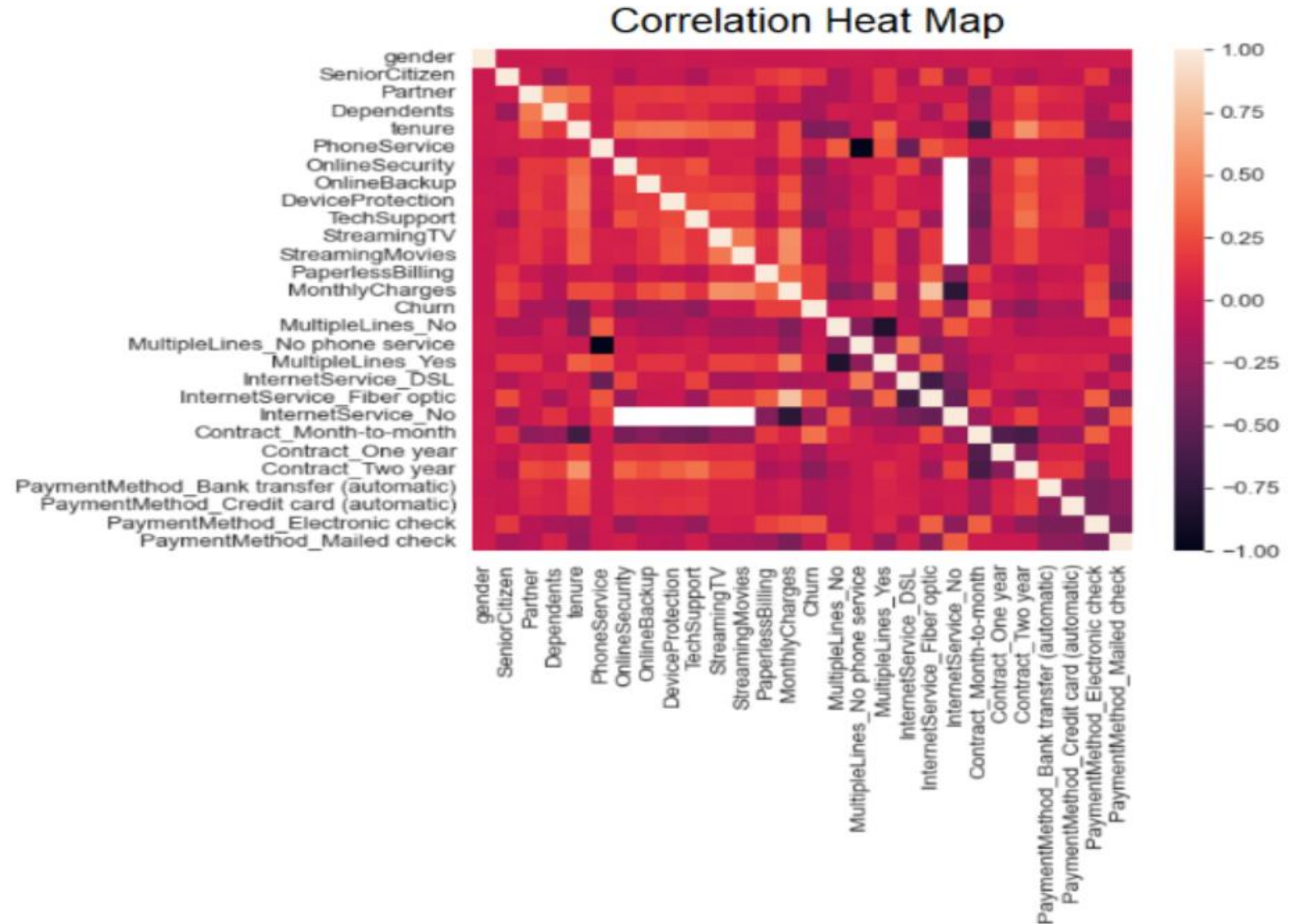
Data Preprocessing

Feature Selection

- Feature Selection was used in this assignment to identify the most relevant columns in the dataset. Selecting only important features helps reduce overfitting and can improve classification metrics.
- Our group selected features by first analyzing the correlation heat map and then applying Chi-square test (chi2) to identify the most important features in the dataset.

Correlation Heat Map

- In this heat map, stronger positive correlations are found among senior citizens, fiber optic customers, month-to-month customers, and electronic check customers.
- There are negative correlation across tenure, online security, tech support, no internet service and two year contract.



Chi-Squared Test

- The image on the lower left shows that One-Hot Encoding was used to convert the categorical columns into binary features that can be processed by the machine learning model. The image on the lower right shows the dependent and independent variable being used with the Chi-squared test (chi2) to evaluate each feature. The parameter k=5 tells the model to select the five most important features.

```
1 from sklearn.preprocessing import OneHotEncoder
2 new_df = df.drop(["customerID", "MonthlyCharges", "TotalCharges", "Churn"], axis=1)
3 cols = new_df.select_dtypes(include=["object"]).columns
4 X = new_df[cols]
5
6 ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
7 transformed = ohe.fit_transform(X)
8
9 encoded_df = pd.DataFrame(transformed, columns=ohe.get_feature_names_out(cols))
10 encoded_df.head()
```

```
1 from sklearn.feature_selection import SelectKBest, chi2
2
3 X = encoded_df
4 y = df["Churn"]
5
6 selector = SelectKBest(chi2, k=5)
7 X_selected = selector.fit_transform(X, y)
8
9 selected_features = X.columns[selector.get_support()]
10 print(selected_features)
```

✓ 0.0s

```
Index(['OnlineSecurity_No', 'TechSupport_No', 'Contract_Month-to-month',
      'Contract_Two year', 'PaymentMethod_Electronic check'],
      dtype='object')
```


Conclusion

- Our dataset did not include no multimedia data and requires no feature creation. The picture below shows the features selected using the Chi-square test. Features with a p-value less than 0.05 are considered statistically significant and therefore important for the machine learning model.

```
1 results = pd.DataFrame({
2     "feature": X.columns,
3     "chi2": selector.scores_,
4     "p_value": selector.pvalues_
5 }).sort_values("chi2", ascending=False)
6
7 print(results)
```

[64] ✓ 0.0s 🗂️ Open 'results' in Data Wrangler

	feature	chi2	p_value
32	Contract_Month-to-month	519.895311	4.459832e-115
34	Contract_Two year	488.578090	2.905390e-108
39	PaymentMethod_Electronic check	426.422767	9.760677e-95
14	OnlineSecurity_No	416.182917	1.653059e-92
23	TechSupport_No	406.117093	2.566524e-90
12	InternetService_Fiber optic	374.476216	1.984260e-83
24	TechSupport_No internet service	286.520193	2.849642e-64
13	InternetService_No	286.520193	2.849642e-64
27	StreamingTV_No internet service	286.520193	2.849642e-64
15	OnlineSecurity_No internet service	286.520193	2.849642e-64

Model Testing and Results

Tested five different models:

Model	Accuracy	Recall	Precision	F1	ROC AUC
Random Forest	0.856	0.714	0.478	0.614	0.824
Decision Tree	0.75	0.497	0.535	0.515	0.766
Logistic Regression	0.80	0.56	0.64	0.60	0.84
Multilayer Perceptron	0.77	0.52	0.56	0.54	0.82
Gradient Boost	0.727	0.791	0.491	0.605	0.747



Why Models Were not Chosen

- Random Forest: had the highest recall but still had **many false negatives**.
- Decision Tree: **missed half the churners**, which fails the business goal.
- Logistic Regression: had higher accuracy, but missed too many churners, including **161 false negatives**.
- Multilayer Perceptron: **weak on the positive churn**.



Best Model

- **Gradient boost** is the most suitable model for our use case.
- We want to catch churners and are okay with accepting false positives.
 - This results in the primary criteria of a high recall on "Yes", with accuracy and other metrics as secondary.
- The model is tuned to increase true positives while keep performance on non-churners acceptable, which is exactly what churn prevention requires.



Deployment

Microservice

- We deployed our model into a FastAPI microservice that is containerized via Docker (making it easier to deploy either on a cloud provider or locally). The source is located at <https://github.com/nexus1976/COMP541/tree/master/deployedmodel>

End point

- The endpoint to ask our model “questions” is at <http://comp541-churn-api-aci.eastus.azurecontainer.io:8000/predict>

Request

- Since the model was trained on the features Contract, PaymentMethod, OnlineSecurity, and TechSupport, a payload specifying these parameters are required to receive and answer from the service.

Response

- Taking in these parameters and in turn utilizing the model, the service will return a YES or NO answer denoting whether the described customer is likely to churn.